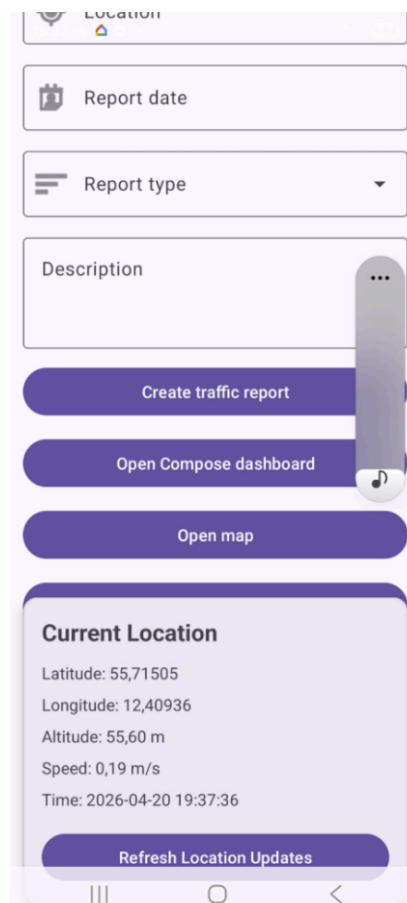
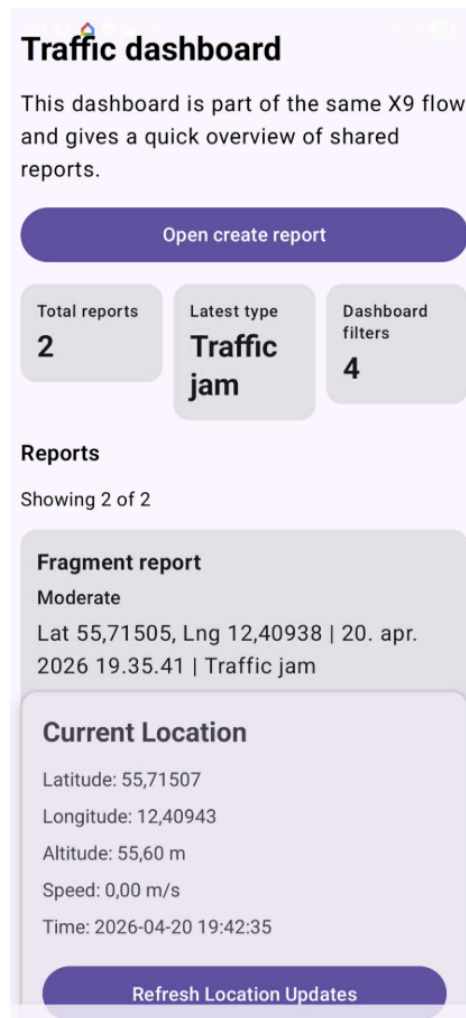


1. Design Choices: Describe your most important design choices regarding classes and layout structures. You can use any digital design software (e.g., Sketch) to illustrate your ideas.

X9 is designed as a traffic reporting app where users can report and view traffic related information such as road incidents, heavy traffic, speed cameras and broken vehicles. The main goal was to make the app simple to use, so users can quickly create a report and see relevant traffic information without too many unnecessary steps.

The app is structured with clear responsibilities. Fragments and a Compose dashboard handle the user interface, while TrafficReportRepository handles communication with Firebase. Shared report data is exposed through TrafficReportViewModel, so different screens such as the report list, map and dashboard can use the same data. This made the project easier to maintain because the Firebase logic is not placed directly inside the UI classes.





The main data model is `TrafficReport`. It represents one traffic report and contains information such as title, type, severity, description, location, image data, timestamps and expiration status. I used a Kotlin data class because it gives a simple structure for storing and passing report data, and it works well with Firebase.

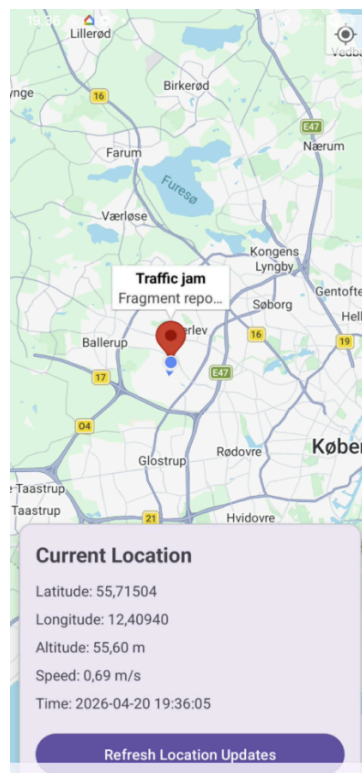
Navigation is handled with the Android Navigation Component and `nav_graph.xml`. The app starts in the main fragment, and from there the user can navigate to screens such as create report, dashboard, map and camera. This made the navigation flow easier to control and understand.

Most screens are built with XML layouts, such as the report form, map screen, details screen and camera screen. I also added a Compose dashboard to show a more modern Android UI approach. The UI was kept simple on purpose, because the app is meant to be used quickly while reporting traffic situations. Material components such as text fields, buttons, cards and RecyclerView rows were used to keep the design consistent.

Firebase is used as the backend. Firebase Authentication handles login, Firebase Realtime Database stores the traffic reports, and Firebase Storage stores uploaded report images. Firebase fits the app well because reports can be updated in real time without the user having to refresh manually.

For location handling, I created a LocationService that uses the Fused Location Provider to get the user's current location. This avoids duplicating location logic in several fragments. The location is used when creating a report, so each report can be connected to a real position on the map.

Reports with latitude and longitude are shown on Google Maps, which makes it easier for users to understand where traffic problems are located. Reports can also include an image. The image is uploaded to Firebase Storage, and the image reference is saved together with the report in Firebase Realtime Database. This gives the report more context and makes the traffic information more useful.



2. User Interfaces: Provide a brief explanation of the user interface of your X9 App.

The user interface in X9 is designed to make traffic reports quick to create and easy to understand. Since the app is about road information, I kept the screens simple and focused on the main tasks.

The app starts on the main screen, where the user can create a quick report, see existing reports and navigate to other parts of the app. If the user is not signed in, the report content is hidden and the user is asked to sign in first. This is necessary because reports are connected to Firebase users.

On the main screen, the user can enter basic report information such as title, location, date, type and description. The severity can be selected as Minor, Moderate or Major. Before saving a report, the app checks that the required fields are filled in and that a GPS location is available. This helps avoid empty or incomplete reports in Firebase.

Reports are shown in a RecyclerView. Each item displays the most important information, such as title, severity, location, date, type and description. The user can tap a report to open the details screen, or remove it with swipe to delete. The create report screen gives the user a more guided way to create a report. It uses the current GPS location, shows the latitude and longitude, and can resolve the address from the coordinates. This makes the report more useful because it includes both an exact map position and a readable location.

The user can also attach an image to a report. The image can be selected from the gallery and previewed before submitting. When the report is saved, the app stores the report in Firebase Realtime Database, uploads the image to Firebase Storage, and saves the image reference on the report. The screen also gives feedback when an image is selected, uploaded or if something fails. The details screen shows the full information for one report, including title, location, date, type, severity, description and image. The list gives a quick overview, while the details screen is used when the user wants to see all information about a report.

The map screen uses Google Maps to show reports with coordinates as markers. When the user taps a marker, the app shows the report title and description. If the user taps the info window, the details screen opens. This makes the reports easier to understand because the user can see where the traffic problem is located. The app also includes an ML Kit camera screen. Here the user can take a photo and analyze it with image labeling.

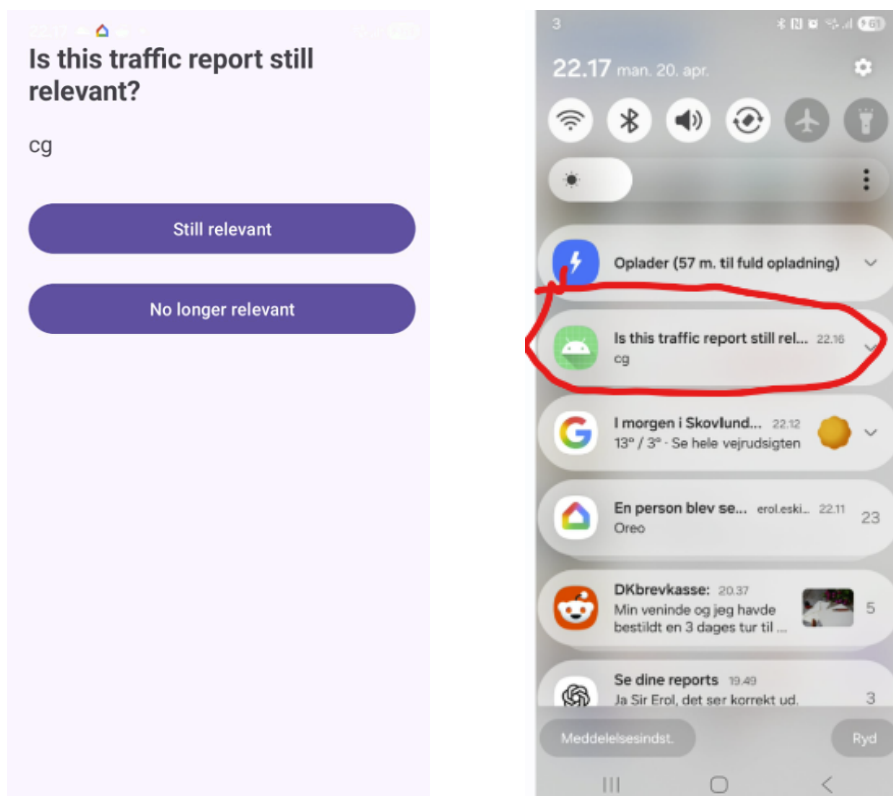
3. Extensions: Discuss extensions and extra features compared to X9 App V10.

As an extra feature, I added a report relevance reminder system. The idea behind this feature is that traffic reports should not stay active forever, because many traffic situations are temporary. For example, heavy traffic, roadwork, accidents or broken vehicles may only be relevant for a short period of time.

When a user creates a new report, the app saves it in Firebase Realtime Database and gives it an expiration time. At the same time, the app uses WorkManager to schedule a background reminder. After two hours, the user receives a notification asking if the report is still relevant.

The notification gives the user two choices. The user can confirm that the report is still relevant, or mark it as no longer relevant. If the report is still relevant, the app updates the report in Firebase and schedules a new reminder. If the report is no longer relevant, the report is marked as expired. Expired reports are not removed from Firebase. Instead, they are filtered out when the app shows reports in the list and on the map. I chose this solution because the data is still kept in the database, but the user only sees reports that are currently useful.

This feature extends the original X9 app because it uses background work, notifications and user feedback to maintain report quality. It makes the app more realistic, because traffic information changes over time and should be kept up to date.



4. Testing and Evaluation: Explain how you tested and evaluated your app to ensure its functionality and usability.

I tested the app by building the project and going through the main features manually. First, I checked that the project could compile without errors.

I tested the login flow by opening the app without being signed in. In that case, the app hides the report content and asks the user to sign in. After logging in with Firebase Auth, the report form and report list become visible.

I also tested creating reports from both the main screen and the create report screen. I checked that the app does not save a report if required fields are missing, or if the GPS location is not ready. When a report is created, it is saved in Firebase and shown in the report list without restarting the app.

The report list was tested by checking that reports are shown in the RecyclerView. I also tested that a report can be opened in the details screen, and that swipe to delete removes it from both the app and Firebase. For the map, I created reports with coordinates and checked that they were shown as markers on Google Maps. I also tested that tapping a marker shows the report information, and that tapping the info window opens the details screen.

Image upload was tested by selecting an image when creating a report. I checked that the image preview is shown, that the image is uploaded, and that it appears on the details screen after the upload.

I tested the ML Kit screen by taking a photo and pressing analyze. I checked that the camera permission is requested, that the photo preview is shown, and that the app displays labels with confidence values. I also checked that the app does not crash if analyze is pressed before taking a photo.

5. Problems: If any issues are encountered during development, document them along with resolutions or workarounds.

One of the biggest problems during development was getting GPS and Google Maps to work correctly. The app had to request location permission, get the user's current location, update it, and use it when creating reports and showing markers on the map. This was challenging because permissions, GPS updates and map integration had to work together.

I solved this by adding permission handling before using GPS. I also used a LocationService and a broadcast receiver, so location updates could be reused in different parts of the app instead of writing the same code several times. The app also checks that a location is available before saving a report.